SpeakPrep: System Design and Architecture

Abhiyan Sainju

April 2026

SpeakPrep — Document 2: System Design + Architecture

Version 1.0 | Author: Abhiyan Sainju | April 2026

Cross-references: Doc 1 (PRD), Doc 3 (Build Guide), Doc 4 (Interview Story)

> **How to use this doc:** This is your technical bible. It explains WHAT the system does and HOW each component works — both at the architecture level and at the concept level. When you're confused about why something is built a certain way, the answer should be here. Read it once before building, reference it constantly during.

TABLE OF CONTENTS

1. System Overview
2. Real-Time Audio: Deep Concepts
3. Voice Activity Detection
4. WebSocket Architecture
5. Python Asyncio + ASGI: Deep Concepts
6. ASR: How Whisper and Deepgram Work
7. LLM Integration + Streaming
8. TTS: How Kokoro Works
9. Full Pipeline Data Flow
10. Database Design
11. Caching Layer: Valkey
12. Authentication + Security
13. Infrastructure Architecture
14. Observability + Monitoring
15. Architecture Decision Records (ADRs)

---

1. System Overview

What the system must do

SpeakPrep's core loop, from the system's perspective:

1. Maintain a persistent WebSocket connection per active session
2. Receive audio binary chunks from the browser
3. Detect when the user finishes speaking (VAD)
4. Transcribe the audio to text (ASR)
5. Generate an intelligent response from the LLM
6. Convert that response to audio (TTS)
7. Stream the audio back to the browser
8. Log everything: latency, transcripts, scores
9. Persist conversation state across reconnects

Every component in the system exists to serve one of these nine functions.

High-Level Architecture Diagram

```
┌──────────────
```

```
    Auth/Users ||    Handler ||       Scoring, Reports ||||||
    Sessions ||      per |session |   (asyncio tasks) |||||└──────────────┘ └──────

                          Internal HTTP ||┌──────────────────┼──────────────────


 FASTER- ||          KOKORO ||            VALKEY ||||
 WHISPER ||          TTS API ||          (Redis) ||||
 (ASR) ||            (port |8880) |      Session ||||
 (port 9000) |└──────────────┘|          Cache |||└──────────────┘ └──────────────┘|

                  EXTERNAL API CALLS ||┌──────────────────────────────────────────

  Groq API (LLM) → api.groq.com ||||
  Deepgram API (streaming ASR) → api.deepgram.com ||||
  Supabase (PostgreSQL + Auth) → [project].supabase.co |||└──────────────────────


                  EXTERNAL SERVICES┌──────────────────────────────────────────

 Cloudflare — DNS, CDN, DDoS, Tunnel ||
 New Relic — APM, metrics, dashboards ||
 Sentry — Error tracking ||
 PostHog — Product analytics + LLM observability ||
 GitHub Actions — CI/CD pipeline |└──────────────────────────────────────────
```

---

2. Real-Time Audio: Deep Concepts

Why Audio is Hard (and What You Need to Know)

Before you write a single line of audio code, you need to understand what you're actually manipulating. Audio feels like a black box until you understand the bytes.

Sample Rate: What It Means

Sample rate is how many times per second we measure the air pressure (amplitude of the sound wave). A sample rate of **16,000 Hz (16 kHz)** means we take 16,000 measurements per second.

Why 16 kHz for voice ASR? - Human speech intelligibility lives in the **100 Hz – 8 kHz frequency range** - Nyquist's theorem: to capture frequencies up to 8 kHz

accurately, you need at least 16,000 samples/second - Higher sample rates (44.1 kHz, 48 kHz) add zero ASR accuracy — Whisper, Deepgram, and every major ASR model internally downsamples to 16 kHz anyway - At 44.1 kHz you'd be sending 2.75x more data for identical results

What a sample actually is: Each sample is a signed 16-bit integer — a number between -32,768 and +32,767. Positive numbers represent compression waves (louder), negative numbers represent rarefaction waves. Zero = silence. Maximum amplitude (clipping) = ±32,767.

PCM: The Raw Format

PCM (Pulse-Code Modulation) is audio in its rawest form — just a sequence of these 16-bit integers, one per sample, in time order. No compression, no headers (mostly), just the numbers.

For our pipeline: **16,000 samples/second × 2 bytes/sample × 1 channel (mono) = 32,000 bytes/second = 32 KB/s.**

This is the format Whisper expects internally. Everything else (MP3, Opus, WebM) is PCM plus compression.

Audio Frames and Chunking

We don't process audio as one continuous stream — we process it in **frames** (fixed-size chunks). Why?

- VAD needs to analyze audio in segments to detect speech vs silence
- Streaming requires sending data incrementally, not waiting for the full recording
- Processing latency must stay bounded — you can't wait for 30 seconds of audio before starting

Frame size decisions:

Frame Size	Samples (16kHz)	Bytes	Latency Added	Tradeoff
10ms	160	320	10ms	Low latency, high overhead (100 frames/sec)
20ms	320	640	20ms	⭐ Best balance — used by WebRTC VAD
30ms	480	960	30ms	Slightly lower overhead
100ms	1,600	3,200	100ms	Too much latency for real-time

Our choice: 20ms frames (640 bytes each). This is the standard for voice AI pipelines.

Audio in the Browser

The browser's **MediaRecorder API** captures microphone audio, but it doesn't give you raw PCM. It gives you encoded audio (WebM/Opus on Chrome, OGG/Opus on Firefox). You have two options:

Option A (simpler, less control): Use MediaRecorder

```
const mediaRecorder = new MediaRecorder(stream, {
  mimeType: 'audio/webm;codecs=opus',
  audioBitsPerSecond: 16000
});
mediaRecorder.ondataavailable = (event) => {
  websocket.send(event.data); // Send Opus/WebM chunks
};
mediaRecorder.start(20); // Get data every 20ms
```

Problem: You get Opus-encoded chunks — you must decode server-side before feeding to Whisper.

Option B (complex, full control): Use Web Audio API

```
const audioContext = new AudioContext({ sampleRate: 16000 });
const source = audioContext.createMediaStreamSource(stream);
const processor = audioContext.createScriptProcessor(320, 1, 1); // 20ms at 16kHz
processor.onaudioprocess = (event) => {
  const pcmData = event.inputBuffer.getChannelData(0); // Float32Array
  const int16Data = float32ToInt16(pcmData); // Convert to PCM Int16
  websocket.send(int16Data.buffer); // Send raw PCM
};
source.connect(processor);
processor.connect(audioContext.destination);
```

Advantage: Server receives raw PCM — no decoding needed, direct to VAD and Whisper.

Our recommendation for SpeakPrep: Use Option B (Web Audio API with ScriptProcessor) for Phase 1. It's more code but eliminates server-side decoding overhead and gives you direct control for VAD.

> **Learn more:** MDN Web Audio API docs, "Audio and Video in HTML5" chapter from O'Reilly

Bandwidth Reality Check

```
Raw PCM:     32,000 bytes/sec = 32 KB/s = 256 kbps per user
Opus 32kbps: 4,000 bytes/sec = 4 KB/s = 32 kbps per user (8:1 compression)
```

```
At 50 concurrent users:
- Raw PCM: 50 × 32 KB/s = 1.6 MB/s upload to server
- Opus: 50 × 4 KB/s = 200 KB/s upload to server

Oracle's free egress bandwidth: ~10 TB/month
At 50 users × 2 hrs/day: 200 KB/s × 3,600s × 365 = ~262 GB/year. Well within limit
```

---

3. Voice Activity Detection

The Problem VAD Solves

If you send audio to Whisper without VAD: 1. Whisper processes silence and **hallucinates** — it generates random text on quiet audio (this affects 40.3% of non-speech segments) 2. You transcribe the user's coughs, breathing, and background noise 3. You can't detect when the user stops talking to trigger transcription

VAD is the gatekeeper: only real speech passes through.

How WebRTC VAD Works (Conceptually)

WebRTC VAD is a **Gaussian Mixture Model (GMM)** classifier. Here's what that means:

1. **Feature extraction:** For each 20ms audio frame, compute a set of spectral features using FFT — essentially "what frequencies are dominant in this chunk of audio?"
2. **GMM classification:** Compare those features against learned probability distributions of "what speech looks like" vs "what noise/silence looks like." A GMM is just a statistical model representing these distributions as overlapping Gaussian curves.
3. **Binary decision:** Output speech=1 or silence=0 based on which distribution is more likely.

Aggressiveness modes (0–3): - Mode 0: Very permissive — catches all speech, including whispers. More false positives (noise classified as speech) - Mode 3: Very strict — only confident speech passes. May clip soft speech or sentence endings

Code for WebRTC VAD:

```python
import webrtcvad
import collections

vad = webrtcvad.Vad(mode=2)  # Aggressiveness 0-3

def is_speech(audio_frame: bytes, sample_rate: int = 16000) -> bool:
    """
    audio_frame must be 10, 20, or 30ms of audio
    At 16kHz with 16-bit mono: 20ms = 640 bytes
```

```
    """
    return vad.is_speech(audio_frame, sample_rate)

def collect_speech_segment(audio_queue, silence_threshold_ms=500):
    """
    Collect audio frames until we detect sustained silence.
    silence_threshold_ms: how long silence must persist before we stop recording
    """
    speech_frames = []
    silence_frames = collections.deque(maxlen=silence_threshold_ms // 20)
    triggered = False

    for frame in audio_queue:
        is_speech_frame = is_speech(frame)

        if not triggered:
            if is_speech_frame:
                triggered = True
                speech_frames.append(frame)
        else:
            speech_frames.append(frame)
            if not is_speech_frame:
                silence_frames.append(frame)
                # Check if we have enough consecutive silence
                if len(silence_frames) == silence_frames.maxlen:
                    if all(not is_speech(f) for f in silence_frames):
                        break  # End of utterance
            else:
                silence_frames.clear()  # Reset silence counter

    return b''.join(speech_frames)
```

Silero VAD: Better for Production

Silero VAD uses a small LSTM neural network trained on massive multilingual data. It outputs a **confidence score (0.0–1.0)** rather than binary. This enables: - **Adaptive thresholding:** Start speaking at 0.5 confidence, stop at 0.35 (hysteresis prevents flickering) - **Noisy environment robustness:** Handles background TV, street noise, HVAC - **Multi-language support:** Works without language-specific tuning

Integration:

```
import torch
model, utils = torch.hub.load(repo_or_dir='snakers4/silero-vad', model='silero_vad
(get_speech_timestamps, _, read_audio, *_) = utils

def silero_detect_speech(audio_tensor):
    """
```

```
    audio_tensor: torch.Tensor of Float32 audio at 16kHz
    Returns: list of {start, end} timestamps where speech occurs
    """
    return get_speech_timestamps(audio_tensor, model, sampling_rate=16000)
```

End-of-Utterance Detection: The Latency Budget

The 500ms silence threshold is the industry default — but it adds 500ms to every turn. If you target 2 seconds total latency, you can't afford to waste 500ms just waiting.

Solutions (in order of complexity):

1. **Aggressive VAD (350ms):** Reduce silence threshold. Risk: clips sentence endings like "...and that's why I think... that approach works." Try 350ms first.
2. **Punctuation streaming:** If using Deepgram streaming, they detect end-of-utterance semantically and return `is_final: true` — often before silence completes.
3. **Semantic endpointing:** Use a small fast model that reads partial transcript and predicts "is this sentence complete?" Can fire 200–300ms before silence ends. This is how LiveKit's EOU model achieves sub-200ms detection.

For SeakPrep Phase 1: Use 400ms silence threshold with WebRTC VAD mode 2. Target 350ms in Phase 2 after measuring actual impact.

> **Learn more:** Silero VAD GitHub (snakers4/silero-vad), WebRTC VAD paper, LiveKit blog on semantic endpointing

---

4. WebSocket Architecture

Protocol-Level Understanding

WebSocket begins as an HTTP request with special headers:

```
Client → Server:
GET /ws/voice/session-123 HTTP/1.1
Host: api.speakprep.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

Server → Client:
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

After this handshake, the connection is **upgraded** — it's no longer HTTP. It's a raw TCP connection with WebSocket framing.

Frame structure:

```
Byte 0: FIN bit (1 = final fragment) + RSV1/2/3 (extensions) + Opcode
        Opcodes: 0x1=text, 0x2=binary, 0x8=close, 0x9=ping, 0xA=pong
Byte 1: MASK bit + Payload length (7 bits, or 16/64 bit extended)
Bytes 2-5: Masking key (→clientserver only — client MUST mask)
Remaining: Payload (XOR'd with masking key if masked)
```

Why do clients have to mask frames? Prevents cache poisoning attacks — if an intermediary proxy sees a WebSocket frame without masking, it might mistake it for an HTTP response and cache it, poisoning the cache for other users.

SpeakPrep WebSocket Message Protocol

Define a clear message schema before building. This prevents protocol drift as the app evolves.

Client → Server messages:

```
# All messages are JSON, except type="audio_chunk" which is binary

# 1. Authentication (first message after connect, must arrive within 5 seconds)
{
  "type": "auth",
  "token": "eyJhbGciOiJIUzI1NiJ9..."
}

# 2. Session start
{
  "type": "session_start",
  "session_id": "550e8400-e29b-41d4-a716",
  "interview_type": "behavioral",  # or "technical", "system_design", "mixed"
  "difficulty": "intermediate",
  "persona": "challenging",
  "duration_minutes": 30
}

# 3. Audio data (BINARY frame, not JSON)
# Raw binary: Float32 PCM samples or Opus-encoded WebM
# Header convention: first 4 bytes = sequence number (uint32)

# 4. Audio end signal (user released push-to-talk, or VAD confirmed end)
{
  "type": "audio_end",
  "sequence": 42,
  "duration_ms": 4200
}
```

```
# 5. Barge-in (user started speaking while AI was talking)
{
  "type": "barge_in"
}

# 6. Heartbeat
{
  "type": "ping",
  "timestamp": 1714500000000
}
```

Server → Client messages:

```
# 1. Auth result
{"type": "auth_result", "success": true, "user_id": "uuid"}

# 2. Session ready
{"type": "session_ready", "first_question": "Let's start..."}

# 3. Transcription (real-time, may update as ASR processes)
{
  "type": "transcript",
  "text": "I led the migration of our monolith to microservices...",
  "is_final": false,  # true = final transcript for this turn
  "confidence": 0.92
}

# 4. AI response text (streamed token by token)
{
  "type": "llm_token",
  "token": "That",
  "sequence": 0
}

# 5. TTS audio chunk (BINARY frame)
# First 4 bytes: sequence number (uint32, big-endian)
# Remaining: Opus audio data
# Client plays chunks in sequence order, buffering if needed

# 6. Turn complete (AI finished responding)
{
  "type": "turn_complete",
  "turn_id": "uuid",
  "latencies": {
    "asr_ms": 412,
    "llm_ttft_ms": 287,
    "tts_first_ms": 309,
    "total_ms": 1248
```

```
  },
  "live_scores": {
    "filler_words": 3,
    "speaking_rate_wpm": 148,
    "estimated_star_compliance": 0.75
  }
}

# 7. Error
{
  "type": "error",
  "code": "ASR_FAILED",
  "message": "Speech recognition failed — please try again",
  "recoverable": true
}

# 8. Session end
{
  "type": "session_end",
  "session_id": "uuid",
  "summary": {...}
}

# 9. Heartbeat response
{"type": "pong", "timestamp": 1714500000000}
```

WebSocket Handler in FastAPI

```
from fastapi import WebSocket, WebSocketDisconnect
from typing import Dict
import asyncio
import json
import uuid

class ConnectionManager:
    """Manages all active WebSocket connections."""

    def __init__(self):
        # Maps session_id → WebSocket
        self.active_connections: Dict[str, WebSocket] = {}

    async def connect(self, websocket: WebSocket, session_id: str):
        await websocket.accept()
        self.active_connections[session_id] = websocket

    async def disconnect(self, session_id: str):
        self.active_connections.pop(session_id, None)
```

```
    async def send_json(self, session_id: str, data: dict):
        ws = self.active_connections.get(session_id)
        if ws:
            await ws.send_json(data)

    async def send_bytes(self, session_id: str, data: bytes):
        ws = self.active_connections.get(session_id)
        if ws:
            await ws.send_bytes(data)

manager = ConnectionManager()

@app.websocket("/ws/voice/{session_id}")
async def voice_endpoint(websocket: WebSocket, session_id: str):
    await manager.connect(websocket, session_id)

    try:
        # Step 1: Authentication (must arrive in 5 seconds)
        auth_msg = await asyncio.wait_for(
            websocket.receive_json(),
            timeout=5.0
        )
        if not await authenticate(auth_msg.get("token")):
            await websocket.close(code=4001, reason="Unauthorized")
            return

        # Step 2: Initialize session
        session = await Session.create(session_id)
        audio_buffer = bytearray()

        # Step 3: Main message loop
        async for message in websocket.iter_bytes():
            if isinstance(message, bytes):
                # Binary = audio data
                audio_buffer.extend(message)

                # Run VAD on latest 20ms frame
                frame = bytes(audio_buffer[-640:])
                if vad.is_speech(frame, 16000):
                    pass  # Continue collecting
                else:
                    if len(audio_buffer) > 3200:  # At least 100ms of audio
                        # End of utterance detected
                        audio_data = bytes(audio_buffer)
                        audio_buffer.clear()

                        # Process in background, continue receiving
                        asyncio.create_task(
```

```
                        process_turn(websocket, session, audio_data)
                    )

    except WebSocketDisconnect:
        await manager.disconnect(session_id)
        await cleanup_session(session_id)
    except asyncio.TimeoutError:
        await websocket.close(code=4002, reason="Auth timeout")
```

Heartbeat + Reconnection

```
# Server-side heartbeat — detect dead connections
async def heartbeat_monitor(websocket: WebSocket, session_id: str):
    while True:
        await asyncio.sleep(20)  # Ping every 20 seconds
        try:
            pong = await asyncio.wait_for(
                websocket.receive_json(),
                timeout=5.0
            )
            # Connection is alive
        except asyncio.TimeoutError:
            # Connection is dead — clean up
            await manager.disconnect(session_id)
            break

// Client-side reconnection with exponential backoff
class VoiceConnection {
    constructor(sessionId) {
        this.sessionId = sessionId;
        this.ws = null;
        this.reconnectAttempts = 0;
        this.maxAttempts = 10;
    }

    async connect() {
        return new Promise((resolve, reject) => {
            this.ws = new WebSocket(`wss://api.speakprep.com/ws/voice/${this.sessi

            this.ws.onopen = () => {
                // Send auth immediately
                this.ws.send(JSON.stringify({
                    type: "auth",
                    token: localStorage.getItem('access_token')
                }));
                this.reconnectAttempts = 0;
                resolve();
            };
```

```
            this.ws.onclose = (event) => {
                if (event.code !== 1000) { // 1000 = normal close
                    this.scheduleReconnect();
                }
            };
        });
    }

    scheduleReconnect() {
        if (this.reconnectAttempts >= this.maxAttempts) return;

        // Exponential backoff: 500ms, 1s, 2s, 4s, 8s, ... capped at 30s
        const delay = Math.min(500 * Math.pow(2, this.reconnectAttempts), 30000);
        // Add jitter: ±25% to prevent thundering herd
        const jitter = delay * 0.25 * (Math.random() * 2 - 1);

        setTimeout(() => {
            this.reconnectAttempts++;
            this.connect();
        }, delay + jitter);
    }
}
```

 Learn more: RFC 6455 (WebSocket Protocol), FastAPI WebSocket docs, MDN WebSocket API

---

5. Python Asyncio + ASGI: Deep Concepts

The Event Loop: What It Actually Is

Python's asyncio event loop is a **scheduler**. Not a thread pool. Not a process pool. A single-threaded scheduler that decides which coroutine to run next.

Here's what happens when you have 3 concurrent voice sessions:

```
Time 0ms: Event loop starts
Time 1ms: Session A connects, starts waiting for audio
Time 2ms: Session B connects, starts waiting for audio
Time 3ms: Session C connects, starts waiting for audio

[All three are now "suspended" — they're waiting for I/O. The event loop is idle.

Time 100ms: Session A sends audio data→
 Event loop wakes session A's coroutine→
 Session A sends audio to Deepgram API→
 Session A hits `await` while waiting for Deepgram response→
 Event loop suspends session A, picks up session B/C if they have work
```

```
Time 150ms: Deepgram responds to Session A→
  Event loop wakes session A again→
  Session A sends transcript to Groq→
  Session A hits `await` again (waiting for Groq)→
  Event loop can advance sessions B and C while A waits
```

This is **cooperative multitasking** — coroutines voluntarily yield at await points. The event loop can't interrupt a coroutine mid-execution (that's preemptive multitasking). If you accidentally write blocking code (like calling requests.get() synchronously), you block the entire event loop.

What `async/await` Actually Means

```python
# This is a COROUTINE FUNCTION (notice the async)
async def process_audio(audio_data: bytes) -> str:
    # This line suspends THIS coroutine and yields control to the event loop
    # The event loop can run other coroutines while we wait for Deepgram
    transcript = await deepgram_client.transcribe(audio_data)

    # When Deepgram responds, the event loop resumes HERE
    return transcript

# A coroutine function returns a COROUTINE OBJECT when called
# It doesn't execute until you await it or create a Task
coro = process_audio(data)  # Nothing happens yet

# Option 1: await directly (sequential)
result = await process_audio(data)

# Option 2: Create a Task (concurrent — runs in background)
task = asyncio.create_task(process_audio(data))
# Do other things...
result = await task  # Wait for it when you need the result
```

The GIL and Why It Doesn't Matter Here

The **Global Interpreter Lock (GIL)** prevents two Python threads from executing Python bytecode simultaneously. Many developers think this makes Python bad for concurrent applications.

For voice AI, the GIL is irrelevant. Here's why:

All the expensive work in our pipeline is I/O-bound or runs outside Python: - **Waiting for Deepgram API response:** OS-level I/O, GIL released - **Waiting for Groq API response:** OS-level I/O, GIL released - **Faster-Whisper inference:** CTranslate2 is C++ with GIL released during inference - **Kokoro TTS inference:** PyTorch releases GIL during tensor operations

The GIL *would* matter if we were doing CPU-bound Python computation (like matrix multiplication in pure Python). We're not.

ASGI: What It Is

WSGI (Flask/Django's standard) is a synchronous protocol: the server calls your app with a request, your app returns a response. Period. No way to push data to clients or maintain persistent connections.

ASGI adds: - Async/await support (coroutines instead of threads) - WebSocket handling (bidirectional persistent connections) - HTTP/2 and HTTP/3 push - Background tasks

```
# ASGI application interface (conceptual)
async def application(scope, receive, send):
    """
    scope: dict with connection info (type, headers, path, etc.)
    receive: async callable to receive messages from client
    send: async callable to send messages to client
    """
    if scope['type'] == 'websocket':
        # Handle WebSocket
        event = await receive()  # e.g., {'type': 'websocket.connect'}
        if event['type'] == 'websocket.connect':
            await send({'type': 'websocket.accept'})

        while True:
            event = await receive()
            if event['type'] == 'websocket.receive':
                # Process message
                await send({
                    'type': 'websocket.send',
                    'text': 'response'
                })
```

FastAPI implements this for you — you write the handler, FastAPI handles the ASGI scaffold.

How Many Concurrent Connections Can We Handle?

```
Single Uvicorn process with uvloop:
- Each WebSocket = ~1 asyncio.Task
- Each Task = ~8 KB stack frame
- 1 GB RAM for connections = ~131,000 connections (theoretical)
- Real-world with voice AI processing: -~5001,000 active sessions
  (limited by external API rate limits, not connection count)

With 4 Gunicorn workers on Oracle ARM (4 OCPUs):
- 4 × 1,000 = ~4,000 concurrent sessions
```

```
- Requires Redis for cross-worker session state sharing
- Well beyond Phase 1 needs (target: 50 concurrent)
```

> **Learn more:** "Python Concurrency with asyncio" by Matthew Fowler (Manning), Real Python asyncio guide, PEP 3156 (asyncio specification), PEP 3333 (WSGI), PEP 3492 (ASGI)

---

6. ASR: How Whisper and Deepgram Work

Whisper's Architecture (What the Model Actually Does)

Whisper is a **Transformer encoder-decoder model** trained on 680,000 hours of audio-transcript pairs collected from the web.

Step 1: Audio preprocessing Raw 16kHz audio → 30-second window (zero-padded) → **log-mel spectrogram** with 80 frequency bins.

What is a mel spectrogram? It's a visual representation of audio that matches human hearing perception. The "mel" scale compresses high frequencies (we can't distinguish 8000 Hz from 8100 Hz well) and expands low frequencies (we can clearly distinguish 100 Hz from 200 Hz). The log transform compresses the amplitude range.

```
# Conceptually, what Whisper's preprocessing does:
import numpy as np
import librosa

audio = load_audio("input.wav", sr=16000)  # Raw PCM

# Compute mel spectrogram
mel = librosa.feature.melspectrogram(
    y=audio, sr=16000, n_mels=80,
    n_fft=400,       # 25ms window
    hop_length=160   # 10ms hop
)
log_mel = librosa.power_to_db(mel)  # Log scale
```

Step 2: Encoder The mel spectrogram passes through 2 Conv1D layers (for local feature extraction) then through N transformer blocks (12 for small, 24 for large). Each transformer block has: - Multi-head self-attention (tokens attend to other tokens) - Feed-forward network (pointwise transformation) - Layer normalization

Output: contextualized audio representations ("what acoustic patterns are here?")

Step 3: Decoder Autoregressively generates text tokens. At each step: 1. Takes previously generated tokens as input 2. Applies causal self-attention (can only see previous tokens, not future ones) 3. Applies cross-attention to encoder output (this is how text "attends" to audio) 4. Outputs probability distribution over vocabulary 5. Sample the most likely token (temperature affects this) 6. Repeat until <|endoftext|> token

Why Whisper has a 30-second limit: The model was trained on 30-second windows. Longer audio requires chunking and merging, which introduces errors at boundaries.

Faster-Whisper and CTranslate2

Original Whisper runs PyTorch with float32 weights. CTranslate2 applies:

1. **INT8 quantization:** Convert weights from 32-bit floats to 8-bit integers. This reduces model size 4x and improves inference speed 2–4x with <1% accuracy loss on most languages.
2. **Layer fusion:** Combine operations that can run together (e.g., matrix multiply + bias add + activation) into single CUDA/CPU kernels.
3. **Batched inference:** Process multiple audio files simultaneously — better GPU utilization.
4. **Optimal memory layout:** Weights stored in memory layout that matches CPU cache patterns.

Performance comparison:

```
Whisper small (PyTorch, float32, CPU): ~18 seconds for 60-second audio
faster-whisper small (CTranslate2, INT8, CPU): ~4.5 seconds (4x faster)
faster-whisper small (CTranslate2, INT8, GPU): ~0.9 seconds (20x faster)
```

Deepgram: How Streaming ASR Works Differently

Deepgram's Nova-3 is a **CTC (Connectionist Temporal Classification)** model, not an encoder-decoder. CTC allows: - Processing audio frames as they arrive (no 30-second window constraint) - Outputting partial results continuously - Sub-300ms latency from speech to transcript

Streaming integration with Deepgram:

```
from deepgram import DeepgramClient, LiveTranscriptionEvents, LiveOptions

async def stream_to_deepgram(audio_queue, result_callback):
    dg = DeepgramClient(api_key=DEEPGRAM_API_KEY)

    options = LiveOptions(
        model="nova-3",
        language="en",
        encoding="linear16",   # Raw PCM
        sample_rate=16000,
        channels=1,
        interim_results=True,    # Get partial results
        endpointing=380,         # 380ms silence = end of utterance
        smart_format=True,       # Add punctuation
        utterance_end_ms="1000" # Alternative endpointing
    )
```

```
    connection = dg.listen.asynclive.v("1")

    @connection.on(LiveTranscriptionEvents.Transcript)
    async def on_transcript(result, **kwargs):
        transcript = result.channel.alternatives[0].transcript
        if result.is_final:
            await result_callback(transcript)

    await connection.start(options)

    async for audio_chunk in audio_queue:
        await connection.send(audio_chunk)

    await connection.finish()
```

ASR Hallucination Prevention

What hallucination looks like:

```
[10 seconds of silence]
Whisper output: "Thank you for watching. Please like and subscribe."

[User clears throat]
Whisper output: "The weather today is sunny with a high of 72 degrees."
```

Multi-layer prevention:

```
async def transcribe_with_validation(audio: bytes) -> Optional[str]:
    # Layer 1: VAD check — don't transcribe if Silero says <0.5 confidence
    if not await silero_vad_check(audio):
        return None

    # Layer 2: Transcribe
    segments, info = whisper_model.transcribe(audio, language="en")
    transcript = " ".join([s.text for s in segments])

    # Layer 3: Check no-speech probability (Whisper outputs this)
    if info.no_speech_prob > 0.6:
        return None

    # Layer 4: Compression ratio (hallucinations are usually repetitive)
    import zlib
    compressed = zlib.compress(transcript.encode())
    compression_ratio = len(transcript.encode()) / len(compressed)
    if compression_ratio > 2.4:  # Very repetitive = likely hallucination
        return None

    # Layer 5: Minimum length (transcribing 5+ seconds with <3 words = suspect)
```

```
    words = transcript.strip().split()
    audio_duration = len(audio) / (16000 * 2)  # seconds
    if audio_duration > 3 and len(words) < 2:
        return None

    return transcript.strip()
```

 Learn more: Whisper paper (arxiv.org/abs/2212.04356), CTranslate2 GitHub, Deepgram streaming docs, "Robust Speech Recognition via Large-Scale Weak Supervision"

---

7. LLM Integration + Streaming

Token Streaming: How It Works

LLM APIs return tokens as they're generated — you don't wait for the complete response. This is critical for voice AI because it lets TTS start synthesizing while the LLM is still generating.

The Groq streaming API (OpenAI-compatible):

```
from groq import AsyncGroq

client = AsyncGroq(api_key=GROQ_API_KEY)

async def stream_llm_response(messages: list) -> AsyncIterator[str]:
    stream = await client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=messages,
        stream=True,
        temperature=0.7,
        max_tokens=300  # Keep responses short for voice
    )

    async for chunk in stream:
        delta = chunk.choices[0].delta
        if delta.content:
            yield delta.content  # Yield one token at a time
```

Sentence-Level TTS Buffering

The key insight: TTS needs complete sentences (or at least complete phrases), not individual tokens. You accumulate tokens until you have a complete sentence, then synthesize it while continuing to collect the next sentence.

```
import re

SENTENCE_ENDINGS = re.compile(r'(?<=[.!?;:])\s+|(?<=\n)\n')
```

```
ABBREVIATIONS = {'Dr', 'Mr', 'Mrs', 'Ms', 'Prof', 'Sr', 'Jr',
                 'vs', 'etc', 'Inc', 'Corp', 'Ltd', 'Co',
                 'e.g', 'i.e', 'a.k.a'}

def is_sentence_boundary(text: str, pos: int) -> bool:
    """Check if position pos is a real sentence boundary."""
    if pos >= len(text):
        return False

    # Check if the period is part of an abbreviation
    before = text[:pos].split()[-1] if text[:pos].split() else ""
    if before.rstrip('.') in ABBREVIATIONS:
        return False

    # Check if it's a decimal number like "3.14"
    if pos > 0 and text[pos-1].isdigit() and pos < len(text) and text[pos].isdigit
        return False

    return True

async def tts_streaming_pipeline(
    llm_stream: AsyncIterator[str],
    websocket: WebSocket,
    kokoro_client
):
    """
    Triple-buffer pipeline:
    - Collect LLM tokens into sentence buffer
    - When sentence complete, synthesize immediately
    - Send TTS audio chunks to client as they're generated
    """
    sentence_buffer = ""
    tts_tasks = []

    async for token in llm_stream:
        sentence_buffer += token

        # Check for sentence boundary
        # Look for ". " or "? " or "! " patterns
        for match in SENTENCE_ENDINGS.finditer(sentence_buffer):
            if is_sentence_boundary(sentence_buffer, match.start()):
                sentence = sentence_buffer[:match.start() + 1].strip()
                sentence_buffer = sentence_buffer[match.end():]

                if sentence:
                    # Fire-and-forget TTS synthesis
                    task = asyncio.create_task(
                        synthesize_and_send(sentence, websocket, kokoro_client)
```

```
                )
                tts_tasks.append(task)

    # Handle remaining buffer (last sentence without terminal punctuation)
    if sentence_buffer.strip():
        task = asyncio.create_task(
            synthesize_and_send(sentence_buffer.strip(), websocket, kokoro_client)
        )
        tts_tasks.append(task)

    # Wait for all TTS tasks to complete
    await asyncio.gather(*tts_tasks)

async def synthesize_and_send(text: str, websocket: WebSocket, kokoro_client):
    """Synthesize one sentence and stream audio chunks to client."""
    async for audio_chunk in kokoro_client.synthesize_stream(text):
        await websocket.send_bytes(audio_chunk)
```

System Prompt Design for Interview Coach

The system prompt is the most important piece of prompt engineering. For a voice AI interview coach:

```
BEHAVIORAL_INTERVIEWER_SYSTEM_PROMPT = """
You are Alex, a senior engineering manager conducting a behavioral interview.
Your job is to assess candidates with the rigor of a real interview.

VOICE RULES (critical for audio):
- Keep ALL responses under 40 words unless asking a follow-up question
- NEVER use bullet points, numbers, or markdown — speak in prose
- Use natural conversational fillers: "I see.", "Go on.", "Interesting."
- Speak in complete sentences that sound natural when read aloud
- After the candidate speaks, acknowledge with 1-2 words before your question

INTERVIEW RULES:
- Ask ONE question at a time. Never two questions in one turn.
- Probe incomplete answers. If Situation/Task/Action/Result is missing any compon
  No Situation: "What was the context? What team or company?"→
  No Result: "What was the actual outcome? Did it ship? What was the impact?"→
  Vague Action: "What did YOU specifically do? What was your decision authority?"
- Push back on clichés: "I'm a team player who..." → "Can you give me a specific
- Challenge weak answers (Challenging/Adversarial mode only):
  "That sounds like what the team did. What would YOU have done differently?"

CANDIDATE CONTEXT:
Name: {candidate_name}
Experience: {years_experience} years
Resume highlights: {resume_summary}
```

```
Target role: {target_role}
Target company: {target_company}

INTERVIEW STATE:
Questions asked: {questions_asked}
Current question type: {current_question_type}
Remaining time: {remaining_minutes} minutes

Start by asking your first behavioral question. Choose based on the candidate's ex
"""
```

Context Window Management

Each Groq API call costs tokens. Keep context tight:

```
class ConversationContext:
    MAX_TURNS_IN_WINDOW = 6  # Keep last 6 turns verbatim
    MAX_SYSTEM_TOKENS = 800
    MAX_HISTORY_TOKENS = 1000

    def build_messages(self, session: Session, new_user_message: str) -> list:
        messages = []

        # 1. System prompt (always first)
        messages.append({
            "role": "system",
            "content": self.build_system_prompt(session)
        })

        # 2. Last N turns of conversation
        recent_turns = session.turns[-self.MAX_TURNS_IN_WINDOW:]
        for turn in recent_turns:
            messages.append({"role": "user", "content": turn.user_transcript})
            messages.append({"role": "assistant", "content": turn.ai_response})

        # 3. Current user message
        messages.append({"role": "user", "content": new_user_message})

        return messages
```

Multi-Provider Fallback with Circuit Breaker

```
import asyncio
from enum import Enum
from dataclasses import dataclass, field
from datetime import datetime, timedelta

class CircuitState(Enum):
```

```
    CLOSED = "closed"      # Normal — requests pass through
    OPEN = "open"          # Failing — requests blocked, use fallback
    HALF_OPEN = "half_open"  # Testing — one request allowed

@dataclass
class CircuitBreaker:
    failure_threshold: int = 5        # Open after 5 failures
    recovery_timeout: int = 60        # Wait 60s before testing
    failure_count: int = 0
    state: CircuitState = CircuitState.CLOSED
    last_failure_time: Optional[datetime] = None

    def call_succeeded(self):
        self.failure_count = 0
        self.state = CircuitState.CLOSED

    def call_failed(self):
        self.failure_count += 1
        self.last_failure_time = datetime.now()
        if self.failure_count >= self.failure_threshold:
            self.state = CircuitState.OPEN

    def can_attempt(self) -> bool:
        if self.state == CircuitState.CLOSED:
            return True
        if self.state == CircuitState.OPEN:
            if datetime.now() - self.last_failure_time > timedelta(seconds=self.re
                self.state = CircuitState.HALF_OPEN
                return True
            return False
        return True  # HALF_OPEN allows one attempt

# Provider chain with circuit breakers
providers = [
    {"name": "groq", "client": groq_client, "breaker": CircuitBreaker()},
    {"name": "cerebras", "client": cerebras_client, "breaker": CircuitBreaker()},
    {"name": "gemini", "client": gemini_client, "breaker": CircuitBreaker()},
]

async def generate_with_fallback(messages: list) -> str:
    for provider in providers:
        if not provider["breaker"].can_attempt():
            continue

        try:
            response = await provider["client"].generate(messages)
            provider["breaker"].call_succeeded()
            return response
```

```
        except (RateLimitError, APIError) as e:
            provider["breaker"].call_failed()
            continue

    raise Exception("All LLM providers exhausted or circuit-broken")
```

 Learn more: Groq API docs, OpenAI streaming docs, "Designing Resilient Systems" (Netflix blog), Circuit Breaker pattern by Martin Fowler

---

8. TTS: How Kokoro Works

Neural TTS Pipeline (General)

Text → Speech is a 3-stage pipeline:

Stage 1: Text Normalization "Dr. Smith spent $3.5M on AI" → "Doctor Smith spent three point five million dollars on AI" Handles numbers, abbreviations, special characters, URLs.

Stage 2: Grapheme-to-Phoneme (G2P) "AI" → /eɪ ˈaɪ/ Converts text characters to phoneme sequences — the actual sounds. This is harder than it looks because English spelling is inconsistent ("through", "tough", "though" — same "ough" but different sounds).

Stage 3: Acoustic Model + Vocoder Phonemes → mel spectrogram → waveform

- Acoustic model: maps phonemes to mel spectrogram (what the frequency-time representation looks like)
- Vocoder: converts mel spectrogram to actual audio waveform (WaveNet → WaveGlow → HiFi-GAN → BigVGAN)

Kokoro Specifically

Kokoro uses **StyleTTS 2 architecture with ISTFTNet vocoder**:

- Only 82M parameters (vs 300M+ for comparable quality models)
- Uses only the decoder path — no diffusion model overhead
- ISTFTNet vocoder: converts frequency domain directly to time domain via inverse STFT — much faster than autoregressive vocoders
- **Real-time factor on CPU: 3–11x** (generates 3–11 seconds of audio for each second of compute)
- **Output: 24 kHz mono audio**

```
# Deploy Kokoro with the Kokoro-FastAPI Docker image
# It exposes an OpenAI-compatible TTS endpoint

import httpx
import asyncio

async def synthesize_speech(text: str, voice: str = "af_heart") -> bytes:
```

```
    """
    Call local Kokoro TTS API.
    Returns raw audio bytes (PCM or WAV depending on config).
    """
    async with httpx.AsyncClient() as client:
        response = await client.post(
            "http://kokoro:8880/v1/audio/speech",
            json={
                "model": "kokoro",
                "input": text,
                "voice": voice,
                "response_format": "wav",
                "speed": 1.0
            },
            timeout=30.0
        )
        return response.content

# For streaming (returns audio as it generates):
async def synthesize_streaming(text: str) -> AsyncIterator[bytes]:
    async with httpx.AsyncClient() as client:
        async with client.stream(
            "POST",
            "http://kokoro:8880/v1/audio/speech",
            json={"model": "kokoro", "input": text, "voice": "af_heart",
                  "response_format": "pcm"},
        ) as response:
            async for chunk in response.aiter_bytes(chunk_size=4096):
                yield chunk
```

Available Kokoro voices: - af_heart — American female, warm - am_adam — American male, conversational - bf_emma — British female, professional - bm_george — British male, authoritative - And 50 more across 8 languages

 Learn more: hexgrad/kokoro on HuggingFace, StyleTTS 2 paper, HiFi-GAN paper, Kokoro-FastAPI GitHub

---

9. Full Pipeline Data Flow

One Complete Voice Turn (Annotated)

This is the journey of a single question-answer exchange, with exact timing targets and failure points.

```
T+0ms: User presses push-to-talk button→
       Client starts MediaRecorder or Web Audio API capture→
       Audio captured at 16kHz, 16-bit, mono
```

```
T+20ms:  First 20ms audio frame (640 bytes) captured→
         Silero VAD check: probability 0.03 (silence) → buffer but don't trigger

T+200ms: User starts speaking "I led the migration..."→
          Silero VAD: probability 0.89 → speech detected→
          Begin collecting frames into speech_buffer

T+3800ms: User finishes speaking→
            500ms of silence detected→
            speech_buffer contains 3.6 seconds of audio (~115 KB PCM)

T+3800ms: ← END OF USER SPEECH | PIPELINE STARTS →

T+3810ms: Audio buffer sent to Deepgram streaming ASR via WebSocket
          [latency target: network RTT ~20ms]

T+4100ms: Deepgram returns partial transcript: "I led the migration of our"→
            Send to client: {"type": "transcript", "text": "...", "is_final": fals
           [Deepgram partial latency: ~300ms from audio end]

T+4250ms: Deepgram returns final transcript: "I led the migration of our monolith
           microservices at ECS Tech, saving the company six months of delivery ti
           {"type": "transcript", "text": "...", "is_final": true}

T+4260ms: LLM request sent to Groq
           Messages include: system prompt + last 6 turns + this transcript
           ~1,800 tokens total input

T+4540ms:  First Groq token arrives: "That"
           [Time-to-first-token: ~280ms — within target]

T+4600ms:  Sentence buffer accumulates: "That's impressive."
           Sentence complete → Send to Kokoro TTS immediately

T+4600ms: Kokoro begins synthesizing "That's impressive."

T+4680ms:  First TTS audio chunk arrives (80ms for 2-word sentence)→
            Send to client: binary WebSocket frame with Opus audio

T+4700ms:  Client begins playing "That's impressive."
           [T+4700ms - T+3800ms = 900ms total latency — under target!]

T-+45405200ms: Groq continues generating next sentence
                "You mentioned microservices — what specific challenges did you en
                during the migration that you personally had to solve?"

T+5100ms: Second sentence complete in LLM stream→
            Kokoro synthesizes while first sentence audio is still playing→
```

```
            Client receives and queues second audio chunk

T+5800ms: LLM stream complete (full response)
           "Done generating"

T+6200ms: All TTS audio delivered to client→
            Client finishes playing all audio

T+6210ms: {"type": "turn_complete", "latencies": {
             "asr_ms": 450,
             "llm_ttft_ms": 280,
             "tts_first_ms": 80,
             "total_ms": 900
           }}

T+6210ms: Background task starts:→
            Full turn saved to PostgreSQL (transcript, response, latencies)→
            Score computed by LLM (async, doesn't block next turn)→
            Live scores computed (filler words, WPM from Deepgram output)→
            Valkey session state updated
```

Total perceived latency: ~900ms (from end of user speech to first word of AI response)

Where Latency Goes Wrong (and How to Fix It)

Bottleneck	How to Detect	Fix
VAD silent threshold too long	asr_ms > 800ms in logs	Reduce silence threshold to 350ms
Deepgram taking >500ms	Log Deepgram-specific latency	Switch to Faster-Whisper locally for this session
Groq TTFT > 400ms	llm_ttft_ms in logs	Provider in high-load period → route to Cerebras
Kokoro TTS > 200ms per sentence	tts_first_ms in logs	Sentence too long (>15 words) → add mid-sentence split
Network RTT > 100ms	Client-side timestamp comparison	User far from Oracle region → consider CDN or regional node

---

10. Database Design

Schema (Full, Production-Grade)

```
-- Enable UUID generation
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";  -- For fuzzy text search

-- ---- USERS ----------------------------------------------------------
CREATE TABLE users (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email               TEXT UNIQUE NOT NULL,
    display_name        TEXT,
    plan                TEXT NOT NULL DEFAULT 'free'  -- 'free', 'pro', 'admin'
                        CHECK (plan IN ('free', 'pro', 'admin')),
    sessions_this_week  INTEGER NOT NULL DEFAULT 0,
    week_reset_at       TIMESTAMPTZ,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_seen_at        TIMESTAMPTZ,
    preferences         JSONB NOT NULL DEFAULT '{}'::jsonb
    -- preferences: {"default_voice": "af_heart", "difficulty": "intermediate", ..
);

-- ---- RESUMES --------------------------------------------------------
CREATE TABLE resumes (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id             UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    filename            TEXT NOT NULL,
    raw_text            TEXT,              -- Extracted PDF text
    parsed_content      JSONB,             -- Structured: {experience, education, skills,
    target_role         TEXT,
    target_company      TEXT,
    experience_years    INTEGER,
    skill_level         TEXT CHECK (skill_level IN ('junior', 'mid', 'senior', 'staff
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    is_active           BOOLEAN NOT NULL DEFAULT true
);

-- ---- SESSIONS -------------------------------------------------------
CREATE TABLE sessions (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id             UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    resume_id           UUID REFERENCES resumes(id),

    -- Session configuration
    interview_type      TEXT NOT NULL CHECK (interview_type IN
                        ('behavioral', 'technical', 'system_design', 'mixed')),
    difficulty          TEXT NOT NULL CHECK (difficulty IN
                        ('beginner', 'intermediate', 'advanced')),
    persona             TEXT NOT NULL CHECK (persona IN
                        ('friendly', 'neutral', 'challenging', 'adversarial')),
    planned_duration_minutes INTEGER NOT NULL DEFAULT 30,
```

```
    -- Session state
    status          TEXT NOT NULL DEFAULT 'active'
                    CHECK (status IN ('active', 'completed', 'abandoned')),
    turn_count      INTEGER NOT NULL DEFAULT 0,

    -- Scores (computed after session)
    overall_score   NUMERIC(4,1),    -- 0.0 to 100.0
    content_score   NUMERIC(4,1),
    communication_score NUMERIC(4,1),
    star_score      NUMERIC(4,1),
    confidence_score NUMERIC(4,1),
    filler_score    NUMERIC(4,1),

    -- Aggregate stats
    total_words_spoken INTEGER DEFAULT 0,
    avg_wpm         NUMERIC(5,1),
    total_filler_words INTEGER DEFAULT 0,
    avg_response_time_ms INTEGER,
    avg_total_latency_ms INTEGER,

    started_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    ended_at        TIMESTAMPTZ,

    -- AI-generated summary
    post_session_summary TEXT,
    top_strengths   TEXT[],
    top_improvements TEXT[]
);

-- ---- TURNS ----------------------------------------------------------
CREATE TABLE turns (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id      UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    sequence        INTEGER NOT NULL,  -- Order within session

    -- Content
    user_transcript TEXT NOT NULL,
    ai_response     TEXT NOT NULL,
    question_type   TEXT,  -- 'behavioral', 'follow_up', 'clarification', 'transit

    -- Scoring
    content_score   NUMERIC(3,1),    -- 1.0 to 5.0
    communication_score NUMERIC(3,1),
    star_score      NUMERIC(3,1),
    confidence_score NUMERIC(3,1),
    filler_score    NUMERIC(3,1),
    overall_score   NUMERIC(3,1),
```

```
    -- AI feedback (JSONB for flexibility)
    feedback          JSONB,
    -- {
    --   "strengths": ["Clear situation context", "Specific action description"],
    --   "improvements": ["Result was vague — no quantified impact"],
    --   "example_better_answer": "...",
    --   "star_analysis": {"situation": true, "task": true, "action": true, "resu
    -- }

    -- Speech analytics
    word_count        INTEGER,
    wpm               NUMERIC(5,1),
    filler_count      INTEGER,
    filler_rate_per_min NUMERIC(4,1),
    response_duration_ms INTEGER,  -- How long user spoke

    -- Latency breakdown (milliseconds)
    vad_latency_ms    INTEGER,
    asr_latency_ms    INTEGER,
    llm_ttft_ms       INTEGER,
    tts_first_ms      INTEGER,
    total_latency_ms  INTEGER,

    -- Model tracking (for debugging + A/B testing)
    asr_model         TEXT,  -- 'deepgram-nova-3', 'whisper-small'
    llm_model         TEXT,  -- 'llama-3.3-70b', 'llama-3.1-8b'
    tts_voice         TEXT,  -- 'af_heart'
    llm_provider      TEXT,  -- 'groq', 'cerebras'

    -- Storage
    audio_url         TEXT,   -- S3/R2 URL if audio stored

    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    UNIQUE(session_id, sequence)
);

-- ---- QUESTION BANK ----------------------------------------------------
CREATE TABLE questions (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    -- Content
    text              TEXT NOT NULL,
    category          TEXT NOT NULL CHECK (category IN
                      ('behavioral', 'technical', 'system_design', 'situational')),
    subcategory       TEXT,  -- 'leadership', 'conflict', 'failure', 'algorithms', 
```

```
    -- Metadata
    difficulty        INTEGER NOT NULL CHECK (difficulty BETWEEN 1 AND 5),
    elo_rating        NUMERIC(6,1) NOT NULL DEFAULT 1200.0,  -- For adaptive difficu

    -- Targeting
    tags              TEXT[] NOT NULL DEFAULT '{}',  -- ['leadership', 'faang', 'se
    companies         TEXT[] DEFAULT '{}',           -- ['google', 'amazon', 'meta']
    roles             TEXT[] DEFAULT '{}',           -- ['swe', 'pm', 'ds']
    min_experience_years INTEGER DEFAULT 0,

    -- Stats (updated by trigger)
    times_asked       INTEGER NOT NULL DEFAULT 0,
    avg_score         NUMERIC(3,1),

    -- Source and quality
    source            TEXT DEFAULT 'curated',  -- 'curated', 'user_contributed'
    is_active         BOOLEAN NOT NULL DEFAULT true,

    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ---- USER QUESTION HISTORY ----------------------------------------
CREATE TABLE user_question_history (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id           UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    question_id       UUID NOT NULL REFERENCES questions(id),
    turn_id           UUID REFERENCES turns(id),
    score             NUMERIC(3,1),
    asked_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    -- ELO update tracking
    user_elo_before NUMERIC(6,1),
    user_elo_after  NUMERIC(6,1),
    question_elo_before NUMERIC(6,1),
    question_elo_after NUMERIC(6,1)
);

-- ---- USER ELO RATINGS ----------------------------------------
CREATE TABLE user_elo_ratings (
    user_id           UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    overall_elo       NUMERIC(6,1) NOT NULL DEFAULT 1200.0,
    behavioral_elo    NUMERIC(6,1) NOT NULL DEFAULT 1200.0,
    technical_elo     NUMERIC(6,1) NOT NULL DEFAULT 1200.0,
    system_design_elo NUMERIC(6,1) NOT NULL DEFAULT 1200.0,
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ---- INDEXES ----------------------------------------
```

```sql
-- User lookups
CREATE INDEX idx_users_email ON users(email);

-- Session queries (user dashboard)
CREATE INDEX idx_sessions_user_id_created ON sessions(user_id, started_at DESC);
CREATE INDEX idx_sessions_status ON sessions(status) WHERE status = 'active';

-- Turn queries (session transcript view)
CREATE INDEX idx_turns_session_id_seq ON turns(session_id, sequence);

-- Question bank queries
CREATE INDEX idx_questions_category_difficulty ON questions(category, difficulty);
CREATE INDEX idx_questions_tags ON questions USING GIN(tags);
CREATE INDEX idx_questions_companies ON questions USING GIN(companies);
CREATE INDEX idx_questions_elo ON questions(elo_rating, is_active) WHERE is_active

-- User question history (avoid repeating questions)
CREATE INDEX idx_uqh_user_question ON user_question_history(user_id, question_id);
CREATE INDEX idx_uqh_user_recent ON user_question_history(user_id, asked_at DESC);

-- Latency analytics
CREATE INDEX idx_turns_latency ON turns(total_latency_ms, created_at DESC);
CREATE INDEX idx_turns_llm_provider ON turns(llm_provider, created_at DESC);
```

Common Query Patterns

```sql
-- User's recent sessions with scores
SELECT s.id, s.interview_type, s.difficulty, s.overall_score,
       s.turn_count, s.started_at, s.ended_at
FROM sessions s
WHERE s.user_id = $1
  AND s.status = 'completed'
ORDER BY s.started_at DESC
LIMIT 20;

-- Score trend for dashboard chart (last 30 sessions)
SELECT
    s.started_at::date as date,
    AVG(s.overall_score) as avg_score,
    AVG(s.content_score) as avg_content,
    AVG(s.communication_score) as avg_comm,
    AVG(s.star_score) as avg_star
FROM sessions s
WHERE s.user_id = $1
  AND s.status = 'completed'
  AND s.started_at > NOW() - INTERVAL '60␣days'
GROUP BY date
ORDER BY date;
```

```sql
-- Find questions user hasn't seen in this category
SELECT q.id, q.text, q.difficulty, q.elo_rating
FROM questions q
WHERE q.category = $1
  AND q.is_active = true
  AND q.id NOT IN (
      SELECT uqh.question_id
      FROM user_question_history uqh
      WHERE uqh.user_id = $2
        AND uqh.asked_at > NOW() - INTERVAL '7 days'
  )
ORDER BY ABS(q.elo_rating - $3)  -- Closest to user's ELO
LIMIT 10;

-- Latency percentiles for monitoring
SELECT
    percentile_cont(0.50) WITHIN GROUP (ORDER BY total_latency_ms) as p50,
    percentile_cont(0.95) WITHIN GROUP (ORDER BY total_latency_ms) as p95,
    percentile_cont(0.99) WITHIN GROUP (ORDER BY total_latency_ms) as p99
FROM turns
WHERE created_at > NOW() - INTERVAL '24 hours';
```

---

11. Caching Layer: Valkey

What Valkey Is and Why We Use It

Valkey is a Redis-compatible in-memory data store. It keeps data in RAM — reads and writes complete in **<1ms** (vs 5–20ms for PostgreSQL). It's the right tool for ephemeral session data that needs to be fast.

Valkey data structures we use:

```python
import valkey.asyncio as valkey

redis = valkey.from_url("redis://valkey:6379", decode_responses=False)

# 1. SESSION STATE (Hash) — Active session data
# Key: session:{session_id}
# TTL: 2 hours (auto-expire if user disconnects)
await redis.hset(f"session:{session_id}", mapping={
    "user_id": user_id,
    "interview_type": "behavioral",
    "turn_count": "0",
    "last_active": str(time.time())
})
await redis.expire(f"session:{session_id}", 7200)  # 2-hour TTL
```

```
# 2. LLM CONTEXT CACHE (String + JSON) — Recent conversation history
# Key: context:{session_id}
# TTL: 1 hour (expire if idle)
context = {"messages": messages_list, "turn_count": 5}
await redis.set(
    f"context:{session_id}",
    json.dumps(context),
    ex=3600  # 1-hour TTL
)
context_str = await redis.get(f"context:{session_id}")

# 3. RATE LIMITING (String + INCR) — Sliding window counter
# Key: ratelimit:{user_id}:{window}
async def check_rate_limit(user_id: str, limit: int = 100) -> bool:
    window = int(time.time() // 3600)  # 1-hour windows
    key = f"ratelimit:{user_id}:{window}"

    count = await redis.incr(key)
    if count == 1:
        await redis.expire(key, 3600)  # Expire at end of window

    return count <= limit

# 4. JWT REVOCATION (Set with TTL)
# Key: revoked_tokens:{user_id}
await redis.sadd(f"revoked_tokens:{user_id}", token_jti)
await redis.expire(f"revoked_tokens:{user_id}", 86400)  # 24 hours

# 5. WEBSOCKET CONNECTION MAP (Hash)
# Useful when running multiple server instances
# Key: ws_connections
await redis.hset("ws_connections", session_id, server_instance_id)

# 6. DAILY SESSION COUNT (String)
# Key: sessions:{user_id}:{date}
date_str = datetime.now().strftime("%Y-%m-%d")
count = await redis.incr(f"sessions:{user_id}:{date_str}")
await redis.expire(f"sessions:{user_id}:{date_str}", 86400)
```

---

12. Authentication + Security

JWT Authentication Flow

```
from datetime import datetime, timedelta
from jose import JWTError, jwt
```

```python
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, Cookie
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

SECRET_KEY = os.environ["JWT_SECRET_KEY"]  # 32+ byte random string
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 15
REFRESH_TOKEN_EXPIRE_DAYS = 7

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
bearer = HTTPBearer()

def create_access_token(user_id: str) -> str:
    payload = {
        "sub": user_id,
        "exp": datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES),
        "type": "access",
        "jti": str(uuid.uuid4())  # Unique token ID for revocation
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def create_refresh_token(user_id: str) -> str:
    payload = {
        "sub": user_id,
        "exp": datetime.utcnow() + timedelta(days=REFRESH_TOKEN_EXPIRE_DAYS),
        "type": "refresh",
        "jti": str(uuid.uuid4())
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

# Login endpoint
@app.post("/api/auth/login")
async def login(credentials: LoginRequest, response: Response):
    user = await verify_user(credentials.email, credentials.password)

    access_token = create_access_token(str(user.id))
    refresh_token = create_refresh_token(str(user.id))

    # Store refresh token in httpOnly cookie (not accessible by JS)
    response.set_cookie(
        key="refresh_token",
        value=refresh_token,
        httponly=True,    # JS cannot read this
        secure=True,      # HTTPS only
        samesite="strict",  # CSRF protection
        max_age=7 * 24 * 3600  # 7 days
    )
```

```
    # Return access token in response body (stored in memory, not localStorage)
    return {"access_token": access_token, "token_type": "bearer"}

# Dependency for protected routes
async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer)
) -> User:
    try:
        payload = jwt.decode(credentials.credentials, SECRET_KEY, algorithms=[ALGO
        user_id = payload.get("sub")
        token_jti = payload.get("jti")

        # Check revocation list
        is_revoked = await redis.sismember(f"revoked_tokens:{user_id}", token_jti)
        if is_revoked:
            raise HTTPException(status_code=401, detail="Token revoked")

        return await get_user_by_id(user_id)
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")
```

Rate Limiting Middleware

```
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware

class RateLimitMiddleware(BaseHTTPMiddleware):

    LIMITS = {
        "/api/auth/": {"requests": 5, "window_seconds": 60},
        "/api/": {"requests": 60, "window_seconds": 60},
        "/ws/": {"connections": 3, "window_seconds": None},  # Concurrent limit
    }

    async def dispatch(self, request: Request, call_next):
        client_ip = request.client.host
        path = request.url.path

        # Determine applicable limit
        for prefix, limit in self.LIMITS.items():
            if path.startswith(prefix):
                key = f"rl:{client_ip}:{prefix}"

                if limit.get("window_seconds"):
                    count = await redis.incr(key)
                    if count == 1:
                        await redis.expire(key, limit["window_seconds"])
```

```
                    if count > limit["requests"]:
                        return JSONResponse(
                            status_code=429,
                            content={"detail": "Rate limit exceeded"},
                            headers={"Retry-After": str(limit["window_seconds"])}
                        )
                break

        return await call_next(request)
```

---

13. Infrastructure Architecture

Docker Compose (Production)

```
# docker-compose.prod.yml

networks:
  frontend:        # Caddy + FastAPI
    driver: bridge
  backend:         # FastAPI + Valkey + databases (isolated)
    driver: bridge
    internal: true  # Cannot reach internet directly

volumes:
  kokoro_models:  # Pre-downloaded TTS models
  valkey_data:    # Persistent Valkey data

services:

  caddy:
    image: caddy:2-alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
    networks: [frontend]
    restart: unless-stopped
    deploy:
      resources:
        limits: {memory: 128m}

  app:
    build:
      context: .
```

```
      dockerfile: Dockerfile
      target: production
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - VALKEY_URL=redis://valkey:6379
      - GROQ_API_KEY=${GROQ_API_KEY}
      - DEEPGRAM_API_KEY=${DEEPGRAM_API_KEY}
      - JWT_SECRET_KEY=${JWT_SECRET_KEY}
      - WHISPER_URL=http://whisper:9000
      - KOKORO_URL=http://kokoro:8880
      - NEW_RELIC_LICENSE_KEY=${NEW_RELIC_LICENSE_KEY}
      - SENTRY_DSN=${SENTRY_DSN}
      - POSTHOG_API_KEY=${POSTHOG_API_KEY}
    networks: [frontend, backend]
    restart: unless-stopped
    depends_on: [valkey, whisper, kokoro]
    deploy:
      resources:
        limits: {memory: 2g}
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/api/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  whisper:
    image: fedirz/faster-whisper-server:latest-cpu
    environment:
      - WHISPER__MODEL=small
      - WHISPER__LANGUAGE=en
    networks: [backend]
    restart: unless-stopped
    deploy:
      resources:
        limits: {memory: 1g, cpus: "1.0"}

  kokoro:
    image: ghcr.io/remsky/kokoro-fastapi-cpu:latest
    volumes:
      - kokoro_models:/app/models
    networks: [backend]
    restart: unless-stopped
    deploy:
      resources:
        limits: {memory: 2g, cpus: "1.5"}

  valkey:
    image: valkey/valkey:8-alpine
```

```
    command: >
      valkey-server
      --maxmemory 512mb
      --maxmemory-policy allkeys-lru
      --save 60 100
    volumes:
      - valkey_data:/data
    networks: [backend]
    restart: unless-stopped
    deploy:
      resources:
        limits: {memory: 600m}

volumes:
  caddy_data:
  kokoro_models:
  valkey_data:
```

Caddyfile

```
# Caddyfile — Automatic HTTPS, WebSocket support, API routing

{
    email your@email.com
}

api.speakprep.com {
    # WebSocket connections (long-lived)
    handle /ws/* {
        reverse_proxy app:8000 {
            flush_interval -1  # Critical: disables buffering for WebSockets
            transport http {
                read_timeout 1h   # Allow 1-hour sessions
                write_timeout 1h
            }
        }
    }

    # REST API
    handle /api/* {
        reverse_proxy app:8000
    }

    # Security headers
    header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains"
        X-Content-Type-Options "nosniff"
        X-Frame-Options "DENY"
```

```
        -Server  # Remove Caddy server header
    }
}

# Web app (deployed separately on Cloudflare Pages — redirect here)
speakprep.com {
    redir https://app.speakprep.com{uri}
}
```

CI/CD Pipeline

```
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
          POSTGRES_DB: speakprep_test
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

      valkey:
        image: valkey/valkey:8-alpine
        options: >-
          --health-cmd "valkey-cli ping"
          --health-interval 10s

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'
```

```
    - name: Install dependencies
      run: pip install -r requirements.txt -r requirements-dev.txt

    - name: Lint (ruff)
      run: ruff check . && ruff format --check .

    - name: Type check (mypy)
      run: mypy app/ --strict

    - name: Run tests
      env:
        DATABASE_URL: postgresql://postgres:test@localhost/speakprep_test
        VALKEY_URL: redis://localhost:6379
        JWT_SECRET_KEY: test-secret-key-minimum-32-characters
      run: pytest tests/ -v --cov=app --cov-report=xml

  build-and-push:
    needs: lint-and-test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          platforms: linux/arm64   # Oracle ARM
          push: true
          tags: ghcr.io/${{ github.repository }}:latest
          cache-from: type=gha
          cache-to: type=gha,mode=max

  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Oracle ARM
        uses: appleboy/ssh-action@v1
```

```
      with:
        host: ${{ secrets.ORACLE_HOST }}
        username: ubuntu
        key: ${{ secrets.ORACLE_SSH_KEY }}
        script: |
          cd ~/speakprep
          git pull origin main
          docker compose -f docker-compose.prod.yml pull
          docker compose -f docker-compose.prod.yml up -d --force-recreate app

          # Run migrations
          docker compose -f docker-compose.prod.yml exec -T app \
            alembic upgrade head

          # Health check
          sleep 10
          curl -f http://localhost:8000/api/health || exit 1

          # Clean up old images
          docker image prune -f
```

---

14. Observability + Monitoring

Health Check Endpoint

```
@app.get("/api/health")
async def health_check() -> dict:
    """
    Returns overall system health.
    Load balancers and monitoring systems call this.
    Return 200 if healthy, 503 if degraded/unhealthy.
    """
    health = {
        "status": "healthy",
        "timestamp": datetime.utcnow().isoformat(),
        "version": os.environ.get("APP_VERSION", "unknown"),
        "checks": {}
    }

    # Check PostgreSQL
    try:
        await db.execute("SELECT␣1")
        health["checks"]["database"] = "healthy"
    except Exception as e:
        health["checks"]["database"] = f"unhealthy:␣{str(e)}"
        health["status"] = "unhealthy"
```

```
    # Check Valkey
    try:
        await redis.ping()
        health["checks"]["cache"] = "healthy"
    except Exception as e:
        health["checks"]["cache"] = f"unhealthy: {str(e)}"
        health["status"] = "degraded"  # Can run without cache

    # Check Kokoro TTS
    try:
        async with httpx.AsyncClient() as client:
            resp = await client.get("http://kokoro:8880/health", timeout=2.0)
            health["checks"]["tts"] = "healthy" if resp.status_code == 200 else "u
    except Exception:
        health["checks"]["tts"] = "unhealthy"
        health["status"] = "degraded"

    # Check Whisper ASR
    try:
        async with httpx.AsyncClient() as client:
            resp = await client.get("http://whisper:9000/health", timeout=2.0)
            health["checks"]["asr"] = "healthy" if resp.status_code == 200 else "u
    except Exception:
        health["checks"]["asr"] = "degraded"  # Deepgram is fallback

    # Current load
    health["active_sessions"] = len(manager.active_connections)

    status_code = 200 if health["status"] == "healthy" else 503
    return JSONResponse(content=health, status_code=status_code)
```

Structured Latency Logging

```
import structlog

log = structlog.get_logger()

async def process_turn(session, audio_data):
    t_start = time.monotonic()

    # ASR
    t_asr_start = time.monotonic()
    transcript = await transcribe(audio_data)
    asr_ms = int((time.monotonic() - t_asr_start) * 1000)

    # LLM
    t_llm_start = time.monotonic()
```

```
llm_ttft_ms = None

async def on_first_token():
    nonlocal llm_ttft_ms
    llm_ttft_ms = int((time.monotonic() - t_llm_start) * 1000)

response = await generate_response(transcript, on_first_token_callback=on_firs
llm_total_ms = int((time.monotonic() - t_llm_start) * 1000)

# TTS
t_tts_start = time.monotonic()
await synthesize_and_stream(response)
tts_ms = int((time.monotonic() - t_tts_start) * 1000)

total_ms = int((time.monotonic() - t_start) * 1000)

# Structured log — shipped to New Relic
log.info(
    "turn_completed",
    session_id=session.id,
    user_id=session.user_id,
    turn_sequence=session.turn_count,
    asr_ms=asr_ms,
    llm_ttft_ms=llm_ttft_ms,
    llm_total_ms=llm_total_ms,
    tts_ms=tts_ms,
    total_ms=total_ms,
    asr_model=ASR_MODEL,
    llm_model=LLM_MODEL,
    llm_provider=LLM_PROVIDER,
)

# Save to DB
await save_turn_latencies(session.id, {
    "asr_ms": asr_ms,
    "llm_ttft_ms": llm_ttft_ms,
    "tts_first_ms": tts_ms,
    "total_ms": total_ms
})

# Alert if > 4 seconds
if total_ms > 4000:
    log.warning("high_latency_turn", total_ms=total_ms, session_id=session.id
```

15. Architecture Decision Records (ADRs)

Each ADR is a permanent record. Written before the decision. Never deleted — only superseded.

ADR-001: WebSocket over REST or gRPC

Date: April 2026 | **Status:** Accepted | **Deciders:** Abhiyan Sainju

Context: Real-time voice requires bidirectional communication. We must choose a protocol.

Options: 1. **REST long-polling** — Client polls every 100ms for new data 2. **Server-Sent Events (SSE)** — One-directional push from server to client 3. **WebSockets** — Full-duplex persistent connection 4. **WebRTC** — Peer-to-peer protocol for real-time audio/video 5. **gRPC streaming** — Binary RPC with bidirectional streaming

Decision: WebSockets

Rationale: - REST polling: 100ms artificial latency, 10x unnecessary requests, no binary data support - SSE: One-directional — can't send audio from client to server - WebRTC: Designed for P2P video; requires complex signaling server; massive overkill - gRPC: Excellent for service-to-service, but browser support requires grpc-web proxy; adds complexity without benefit - WebSockets: Bidirectional, binary-capable, universally supported in browsers and React Native, 2-byte frame overhead vs 700+ bytes for HTTP

Consequences: - Must implement reconnection logic (WebSocket connections can drop) - Load balancers need sticky sessions for multi-server deployment - More complex debugging than REST (can't use curl — need wscat or browser dev tools) - ✅ Enables real-time streaming in both directions with minimal overhead

---

ADR-002: Web-First (No Mobile in Phase 1)

Date: April 2026 | **Status:** Accepted

Context: Should we build mobile (React Native) in Phase 1 alongside web?

Options: 1. Build web + mobile simultaneously 2. Web-first, mobile in Phase 2 3. Mobile-first (PWA or native)

Decision: Web-first, mobile in Phase 2

Rationale: - React Native audio APIs (expo−av, expo−audio) have documented instability issues, particularly on iOS for real-time streaming - Mobile builds take significantly longer per iteration (10-minute EAS build vs instant Vite dev server) - Browser Web Audio API is mature and gives precise control over audio capture - Can test and iterate 3–5x faster on web - Most interview prep happens at a desk (desktop/laptop browser)

Consequences: - Phase 1 users limited to desktop browsers - Chrome, Firefox, Safari, Edge are primary targets - Mobile is Phase 2 — architecture must not require rewrite to add it

---

ADR-003: Groq as Primary LLM Provider

Date: April 2026 | **Status:** Accepted

Context: Which LLM provider to use for generating interview responses?

Decision: Groq (Llama 3.3 70B) as primary, Cerebras as secondary fallback, Gemini Flash-Lite as tertiary

Rationale: - Groq: Sub-100ms TTFT via LPU hardware. At 315 tokens/sec, a 50-word AI response takes ~0.2 seconds to generate. This is the single most important metric for voice AI. - OpenAI GPT-4o: 300–800ms TTFT — too slow for natural conversation feel - Self-hosted Llama (Ollama): 5–15 tokens/sec on ARM CPU — far too slow for real-time voice - Cerebras: 2,500 tokens/sec, 80–150ms TTFT — excellent backup. Fewer model options. - Groq free tier: 30 RPM, 500K tokens/day — sufficient for development and early users

Assumptions (must validate): - Groq free tier remains available as user base grows - Llama 3.3 70B quality is sufficient for interview coaching (vs GPT-4o)

Consequences: - Must implement multi-provider fallback for rate limit handling - Tied to Groq's model availability — may need to update model string periodically - OpenAI-compatible API format allows switching providers with minimal code change

---

ADR-004: Kokoro TTS (Self-Hosted) as Primary TTS

Date: April 2026 | **Status:** Accepted

Context: Which TTS engine for voice responses?

Decision: Kokoro TTS self-hosted via Kokoro-FastAPI Docker image

Rationale: - ElevenLabs: 10,000 chars/month free = ~10 minutes of audio. Unsustainable at any scale. - Cartesia: 20,000 chars/month + $5/month for usable limits. Good for later. - Google Cloud TTS: 4M chars/month free, but quality is 6/10 on Standard voices. - Kokoro: Apache 2.0, 8.5/10 quality (#1 HuggingFace TTS Arena), runs on CPU, unlimited usage, 54 voices, <300ms latency per sentence.

Consequences: - Consumes ~1–2 GB RAM on Oracle ARM (within 24 GB budget) - No voice cloning in Phase 1 (fixed voices only) - Must maintain Docker deployment — no vendor SLA - **When paid users demand higher quality:** Add Cartesia at $0.002/char as Pro upgrade

---

ADR-005: Deepgram for Streaming ASR (over Faster-Whisper-only)

Date: April 2026 | **Status:** Accepted

Context: Use external streaming ASR (Deepgram) or self-hosted Whisper?

Decision: Deepgram as primary (uses $200 free credit), Faster-Whisper as fallback

Rationale: - Faster-Whisper requires batch processing — collect full audio, then transcribe. Adds 100–500ms latency vs Deepgram's native streaming. - Deepgram returns partial transcripts 300ms after speech starts (before user finishes talking). This enables "transcript-while-speaking" UI feature. - Deepgram's nova-3 model achieves <5% WER vs Whisper's 8–10% on conversational speech. - $200 free credit = ~26,000 minutes. At 20 minutes/user × 100 test users = 2,000 minutes. Credit lasts well through Phase 1.

Assumptions: - $200 credit provides sufficient runway through Phase 1 launch - Self-hosted Faster-Whisper covers fallback adequately

Consequences: - External API dependency for primary ASR path - When Deepgram credit exhausted: migrate to self-hosted Faster-Whisper (already built)

ADR-006: Supabase for Phase 1 Database + Auth

Date: April 2026 | **Status:** Accepted (Phase 1 only, revisit at 500 users)

Decision: Use Supabase free tier for PostgreSQL + Auth

Rationale: - Supabase bundles: PostgreSQL + JWT auth + Row-Level Security + Storage + real-time - Self-building auth (password hashing, JWT issuance, token refresh, email verification) takes 2–3 weeks - Supabase saves those weeks for Phase 1 core product work - 500 MB storage is sufficient for text-only session data (no audio storage) - Pause prevention: automated cron job hitting /api/health every 6 days

Consequences: - 500 MB storage limit — must migrate to self-hosted PostgreSQL on Oracle by ~500 users - Supabase as single point of failure — acceptable for Phase 1 - Row-Level Security policies should be written from day 1 so migration doesn't require schema rewrite

---

End of Document 2 — System Design + Architecture Next: Document 3 — Phase-by-Phase Build Guide with Learning Curriculum